



Dipartimento di INGEGNERIA INFORMATICA,
MODELLISTICA, ELETTRONICA E SISTEMISTICA

Corso di Laurea in Ingegneria Informatica

AIOps e Model Context Protocol

Bridge architetturale tra LLM e hypervisor

Relatore:

Prof. Sergio Greco

Candidato:

Michele Arnoni

Matricola: 220121

Correlatore:

Prof. Lucio La Cava

Anno Accademico 2025/2026

Indice

1	Introduzione	1
1.1	Contesto e scenario generale	1
1.1.1	Hypervisor	1
1.1.2	Model Context Protocol (MCP)	1
1.2	Problematiche che si propone di risolvere	1
1.3	Obiettivi della tesi	2
2	Background	3
2.1	Evoluzione delle ITOps e AIOps	3
2.2	Infrastructure as Code (IaC) e API REST	4
2.3	Interfacce per LLM	5
2.4	Limiti degli approcci attuali	6
2.5	Rassegna delle tecnologie alternative	6
3	Analisi dei Requisiti	8
3.1	Requisiti Funzionali	8
3.2	Requisiti Non Funzionali	8
4	Progettazione e scelte architetturali	9
4.1	Architettura	9
4.2	Scelte progettuali	9
5	Sviluppo e implementazione	10
5.1	Funzioni chiave	10
5.2	Sfide tecniche	10
6	Testing e valutazione sperimentale	11
6.1	Soddisfazione dei criteri di test	11
6.2	Metriche di performance e benchmark	11
6.3	Analisi dei risultati	11

7 Conclusioni e sviluppi futuri	12
7.1 Conclusioni oggettive	12
7.2 Sviluppi futuri	12
Bibliografia	13

Sommario

L'integrazione dei *Large Language Model (LLM)* può offrire nuove opportunità per la semplificazione nella gestione e nella manutenzione di infrastrutture server IT.

In particolare, tramite l'emergente protocollo *MCP Server (Model Context Protocol)*, è possibile fornire ad un *LLM* un layer di comunicazione standardizzato con un sistema IT. Questo approccio abilita interazioni basate sul linguaggio umano con tutta una serie di vantaggi in termini di semplicità e immediatezza operativa per task che tradizionalmente richiederebbero approcci più impegnativi e complessi.

Tuttavia, è importante considerare la natura aleatoria e non deterministica intrinseca di un sistema basato su AI, pertanto questo lavoro di tesi illustra lo sviluppo di un *server MCP* che permette il relazionarsi di un *LLM* con un *Hypervisor Prox-mox*, con annessa valutazione dei limiti e benefici che tale architettura comporta, ponendo una rigorosa attenzione alla mitigazione dei rischi e alla sicurezza. Attraverso la validazione di questo Proof of Concept, il lavoro dimostra come l'impiego del protocollo *MCP*, unito ad adeguati vincoli architetturali, permetta di governare l'esecuzione di task potenzialmente distruttive in modo controllato e affidabile.

Capitolo 1

Introduzione

1.1 Contesto e scenario generale

1.1.1 Hypervisor

In ambito infrastrutturale IT, una macchina di tipo server opera grazie all'impiego di sistemi operativi specialistici, noti come *Hypervisor* [1], specializzati nella virtualizzazione multipla di ambienti disaccoppiati tra loro e su cui gireranno i vari servizi.

1.1.2 Model Context Protocol (MCP)

Un qualsiasi *LLM* può interagire con software classici interpretandoli come tools e instaurando con essi una comunicazione simil API, ma molto più standardizzata. In questo scenario l'azienda *Anthropic* ha sviluppato un protocollo chiamato *MCP* [2], che permette interazioni strutturate basate sulla logica di un sistema client-server, rispettivamente *LLM* e il layer *MCP*.

Attualmente il progetto *MCP* è opensource ed è stato donato da *Anthropic*, per garantirne l'indipendenza, alla *Agentic AI Foundation* [3] [4], un fondo gestito dalla *Linux Foundation*.

1.2 Problematiche che si propone di risolvere

La gestione di un *hypervisor* richiede competenze sistemistiche verticali e l'uso di interfacce GUI o CLI spesso complesse e poco user-friendly. In questo lavoro di tesi si propone di realizzare e valutare un *MCP server* con il ruolo di bridge architetturale tra un *LLM* e un hypervisor aggiungendo così una nuova interfaccia per l'utente basata sul linguaggio naturale.

1.3 Obiettivi della tesi

L'architettura non si limita ad interfacciare un *LLM* ad un hypervisor, ma prevede soluzioni volte a irrobustire questa interfaccia con scelte di design che gestiscono operazioni potenzialmente distruttive e mantengono il controllo sulle azioni dell'*LLM*.

Capitolo 2

Background

2.1 Evoluzione delle ITOps e AIOps

Tutte le operazioni volte alla gestione delle infrastrutture IT, quali manutenzione, ottimizzazione e pianificazione delle operazioni strategiche a lungo termine, sono descritte dal termine *ITOps* (*IT Operations*) [5], termine ormai consolidato nel settore, ma che ha subito profonde trasformazioni con l'avvento del cloud computing e la crescente complessità dei sistemi distribuiti.

Tradizionalmente, le attività di ITOps venivano gestite in modo prevalentemente manuale. Gli operatori facevano affidamento su cruscotti di monitoraggio statici e sistemi di alert automatizzati basati su parametri preimpostati.

Con l'esplosione dei dati generati dalle infrastrutture moderne (log, metriche, tracce) e la transizione verso architetture a microservizi, questo approccio si è rivelato difficilmente scalabile e con un'inefficienza direttamente proporzionale alle dimensioni del sistema.

La frammentazione dei servizi e l'enorme volume di alert possono rendere molto rumorosa e lenta l'identificazione della causa radice (*root cause analysis*) di un disservizio.

Per rispondere a queste sfide, negli ultimi anni si è assistito all'introduzione delle *AIOps* (*Artificial Intelligence for IT Operations*) [6]. Questo termine, coniato da *Gartner* [7], una delle più importanti e influenti società di ricerca e analisi nel campo IT, descrive come l'impiego di software di monitoring o interazione basato su AI all'interno dei flussi di lavoro delle ITOps possa portare a benefici concreti in termini di incremento dell'immediatezza operativa e di gestione della grossa mole di alert.

L'obiettivo principale delle AIOps è aumentare le performance di identificazione e risoluzione di disservizi in tempi più rapidi e di incapsulare automazioni complesse

ed eterogenee in comandi impartiti tramite il linguaggio naturale.

Le soluzioni AIOps operano tipicamente attraverso diverse fasi:

- **Raccolta e aggregazione:** centralizzazione di grandi volumi di dati eterogenei provenienti da molteplici sorgenti dell'infrastruttura.
- **Analisi e rilevamento:** utilizzo di algoritmi di machine learning per l'interpretazione dei dati in tempo reale.
- **Correlazione:** raggruppamento di eventi correlati per ridurre il rumore di fondo ed evitare la duplicazione delle segnalazioni, demoltiplicando gli alert.
- **Risoluzione automatica:** attivazione immediata di workflow automatizzati per mitigare o risolvere il problema senza l'intervento umano diretto o invio di proposte di risoluzione automatizzate con scelta a cura di un umano per le problematiche più critiche.

L'evoluzione verso le AIOps rappresenta quindi un passo fondamentale per garantire la resilienza e l'efficienza dei sistemi IT moderni, ponendo le basi per una gestione infrastrutturale semplificata e strutturata.

2.2 Infrastructure as Code (IaC) e API REST

L'*IaC* (*Infrastructure as Code*) [8] è una pratica ingegneristica che prevede di trattare una infrastruttura IT come un software, con tutti i vantaggi che ne conseguono.

Il workflow di un team IT basato sul paradigma *IaC* si compone delle seguenti fasi:

- **Writing:** scrittura dei file di configurazione architetturale tramite linguaggi tipo *HCL*, *YAML* o *JSON*.
- **Versioning:** i rilasci del codice sorgente vengono strutturati tramite sistemi di controllo delle versioni come *Git*, incorporando anche test di controllo automatici ad ogni nuova versione.
- **Provisioning:** un engine di automazione legge i file di configurazione e ne applica le specifiche in maniera idempotente, ovvero senza duplicare azioni già applicate in precedenza o riassegnare ripetutamente le stesse risorse.
- **Distributing:** si applicano script per la distribuzione dell'infrastruttura in vari ambienti assicurando coerenza e stabilità grazie all'uso dei file sorgente già redatti e testati nella fase di Writing.

È facile intuire come questo approccio, più strutturato e di impronta fortemente ingegneristica, porti concreti vantaggi in termini di controllo del sistema, robustezza, manutenibilità, resilienza all'errore umano, riproducibilità delle configurazioni e scalabilità del sistema.

Ponendo particolare attenzione alla fase di Provisioning e alle tecnologie che la popolano, è possibile imbattersi in strumenti di automazione specifici al sistema su cui operano, come *AWS CloudFormation*, *Azure Resource Manager* o *Google Cloud Deployment Manager*, oppure in strumenti universali come *Terraform* [9]. Questi strumenti si interfacciano con il sistema tramite *API (Application Programming Interface)* permettendo interazioni dirette e programmate ed evitando le interazioni manuali tramite console.

Questo lavoro di tesi vuole evolvere questi strumenti grazie all'AI, rendendoli più flessibili e potenti, integrando un *MCP* come layer di comunicazione verso *API REST* permettendo ad un LLM di dialogare con i sistemi IT.

2.3 Interfacce per LLM

I Large Language Model sono di base incapaci di interagire con software esterni, per superare questi limiti architetturali ogni provider ha sviluppato in autonomia un proprio paradigma di interazione tra cui spicca il modello *ReAct (Reasoning and Acting)* introdotto da Yao et al. [10]. Questo pattern istruisce il modello a produrre un output che alterna in modo iterativo tracce di ragionamento logico (Reasoning) ad azioni specifiche (Acting) rivolte a sistemi esterni, permettendo all'agente di aggiornare dinamicamente il proprio contesto.

L'implementazione tecnica del pattern ReAct si concretizza tramite il meccanismo del *Tool Calling* (o *Function Calling*), un'idea che prevede di iniettare la lista strutturata in formato JSON dei tools a disposizione dell'IA con tanto di descrizione di ogni funzione chiamabile insieme ai suoi parametri.

La pipeline di funzionamento si compone quindi nei seguenti step:

- **Decisione:** l'agente determina che per soddisfare una richiesta è necessario interagire con un tool
- **Invocazione:** l'LLM genera un *payload* JSON contenente l'intenzione di invocare un *tool* specifico e i relativi argomenti tipizzati.
- **Esecuzione:** l'applicazione chiamante intercetta, gestisce la richiesta e re-inietta il risultato nel contesto dell'LLM.

- **Sintesi** l’LLM riprende con la sua generazione per terminare la sintetizzazione della risposta

2.4 Limiti degli approcci attuali

Sebbene il *Tool Calling* offra un’interfaccia tra i modelli generativi e i sistemi IT, la sua implementazione non filtrata in ambienti di produzione critici solleva sfide ingegneristiche non trascurabili. Dal momento in cui un sistema dipendente da un hypervisor *bare-metal* come Proxmox VE è spesso impiegato in ambito enterprise l’esecuzione di comandi errati può arrecare danni economici importanti.

La principale causa di comandi distruttivi deriva dal fenomeno delle ”allucinazioni” che può colpire un qualsiasi LLM o da un prompt ambiguo non deterministicamente eseguibile.

Un *Large Language Model*, a causa della sua natura probabilistica, potrebbe generare *payload* JSON formalmente validi ma semanticamente dannosi.

Negli approcci tradizionali risulta complesso implementare un rigoroso *Principio del Minimo Privilegio*, poiché la logica di validazione e filtraggio delle intenzioni pericolose ricade interamente sul client applicativo.

In secondo luogo, l’implementazione *custom* dei *tool* crea un forte accoppiamento (*tight coupling*) tra la logica decisionale dell’agente e le API dell’infrastruttura con il rischio di trovarsi con un’architettura poco scalabile e molto rigida.

Infine, le integrazioni convenzionali mancano di un meccanismo per l’assegnazione dinamica delle funzionalità in quanto solitamente l’elenco degli strumenti e delle risorse disponibili deve essere inserito staticamente nel prompt di sistema (*hardcoded*). Questo impedisce all’infrastruttura di esporre o revocare dinamicamente le capacità operative in base allo stato corrente del server o ai privilegi dell’utente richiedente. Questa rigidità strutturale e l’assenza di un perimetro di sicurezza delimitato evidenziano la necessità di un layer di astrazione intermedio tra l’intelligenza artificiale e le API di sistema.

2.5 Rassegna delle tecnologie alternative

Per ovviare ai limiti del pattern ReAct e del tool calling in un contesto di produzione critica, esistono diverse tecnologie e approcci alternativi che potrebbero essere considerati. Alcune di queste includono:

API Gateways: un gateway API può fungere da intermediario tra l’LLM e le API di sistema, garantendo un livello di sicurezza aggiuntivo e consentendo il

controllo di accesso e l'autenticazione. Gli API gateways supportano anche l'aggregazione di dati da più servizi, il routing di chiamate e la gestione di errori, riducendo la complessità di sviluppo e l'accoppiamento tra l'agente e le API.

Orchestratori e Automazioni: strumenti come Ansible, Puppet, e SaltStack, possono essere utilizzati per orchestrazione di flussi di lavoro e automazione di task. Questi strumenti possono essere utilizzati per creare pipeline di lavoro predefinite e sicure per l'invocazione di specifici comandi sul sistema.

Microservizi e FaaS (Function as a Service): implementare una serie di microservizi o funzioni specifiche che gestiscono le operazioni critiche può rendere il sistema più modulare e sicuro. Questo approccio permette di separare l'aspetto decisionale del modello di linguaggio dalle operazioni concrete sul sistema, rendendo più facile implementare il principio del minimo privilegio e un accesso condizionale. È però importante considerare che il demando della traduzione delle intenzioni dell'IA e la validazione dei payload JSON interamente al codice *custom* comporta un notevole onere di sviluppo e manutenzione.

Middleware di sicurezza: esistono soluzioni specifiche di middleware come Keycloak o OAuth2 Proxy che possono essere utilizzate per aggiungere una layer di sicurezza aggiuntivo tra l'LLM e le API di sistema. Questi strumenti possono gestire l'accesso e l'autenticazione di sicurezza, riducendo il rischio di allucinazioni o comportamenti ambigui.

Sistemi a Plugin Proprietari: i principali provider di LLM di frontiera hanno tentato di risolvere il problema incorporando protocolli proprietari (come gli ormai deprecati OpenAI Plugins), che permettevano al modello di interrogare API esterne tramite la lettura di file di specifica (come OpenAPI). Tali soluzioni, seppur funzionali, generano un forte *vendor lock-in* e risultano inadatte ad architetture on-premise chiuse come i cluster Proxmox.

Capitolo 3

Analisi dei Requisiti

3.1 Requisiti Funzionali

3.2 Requisiti Non Funzionali

Capitolo 4

Progettazione e scelte architettoniche

4.1 Architettura

4.2 Scelte progettuali

Capitolo 5

Sviluppo e implementazione

5.1 Funzioni chiave

5.2 Sfide tecniche

Capitolo 6

Testing e valutazione sperimentale

6.1 Soddisfazione dei criteri di test

6.2 Metriche di performance e benchmark

6.3 Analisi dei risultati

Capitolo 7

Conclusioni e sviluppi futuri

7.1 Conclusioni oggettive

7.2 Sviluppi futuri

Bibliografia

- [1] IBM - Stephanie Susnjara e Ian Smalley. *hypervisor overview*. URL: <https://www.ibm.com/it-it/think/topics/hypervisors>.
- [2] Anthropic. *MCP announcement*. Nov. 2024. URL: <https://www.anthropic.com/news/model-context-protocol>.
- [3] Linux Foundation. *MCP Agentic Ai Foundation*. URL: <https://aaif.io/projects/model-context-protocol>.
- [4] Linux Foundation. *MCP Project Page*. URL: <https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro>.
- [5] IBM. *ITOps overview*. URL: <https://www.ibm.com/it-it/think/topics/it-operations>.
- [6] IBM. *AIOps overview*. URL: <https://www.ibm.com/it-it/think/topics/aiops>.
- [7] Gartner. *AIOps abstract insights*. URL: <https://www.gartner.com/en/documents/5398763>.
- [8] IBM - Jim Holdsworth e Annie Badman. *Infrastructure as Code overview*. URL: <https://www.ibm.com/it-it/think/topics/infrastructure-as-code>.
- [9] IBM/HashiCorp - Derek Robertson - Matthew Kosinski - Gregg Lindemulder. *Terraform topic*. URL: <https://www.ibm.com/it-it/think/topics/terraform>.
- [10] Shunyu Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *The Eleventh International Conference on Learning Representations (ICLR)*. 2023. URL: <https://arxiv.org/abs/2210.03629>.